

Tutorial-1

EEE/CS/ECE/INST F241-Microprocessor Programming & Interfacing

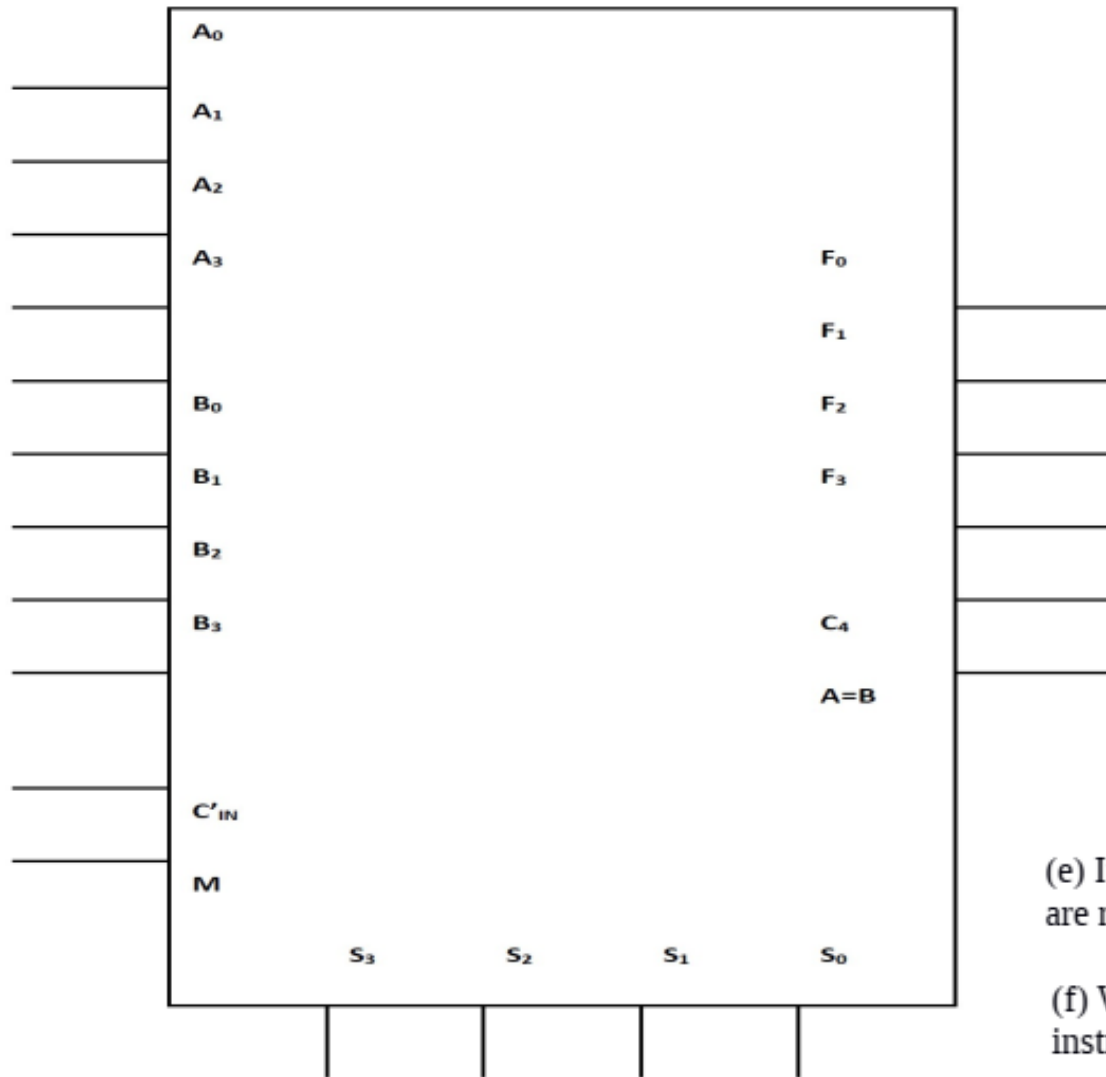
Q.1 How do you determine whether if a microprocessor is 8 bits or 16-bits or 32-bits?

Sol: Size of ALU (i.e. Maximum size of data on which the arithmetic or logical operation can be performed)

Q.2 What is the order in which the microprocessors bus signals (Address, Data and Control) are activated?

Sol: Address, Control, Data

Q3. Given the following ALU chip (ALU 181)



→ S₃ – S₀

1. Selects ALU operation to perform

→ M

1. 1-Logical
2. 0-Arithmetic

→ CIN

→ C₄ along with A= B can be used for unsigned comparison

(a) What will be the size of CPU built around this ALU

4- bit

(b) If other than ALU operations- the processor performs

- Load data from mem to reg
- Store data from reg to mem
- Move data from reg to reg
- Unconditional Branch
- 4 Conditional branches
- 1 Call
- 1 Return

How many bits are required to represent registers in the machine code?

4- bit

And Instruction is of the format – Opcode dst,src , All ALU operations are performed on registers, There are totally 16 Registers R0- R15 .

(c) What type of architecture (RISC/CISC) does the processor implement?

RISC

(d) What will be the normal size of the instruction?

2 bytes

(e) If the Processor can support 64KB of memory and Memory is byte organised- How many address are required?

16 address lines

(f) What will the number by which the Program Counter be incremented to support the normal sized instruction?

2

(g) Can you think of any other major digital modules that the processor may require?

Decoder, control logic, clocking circuitry

Q.4 What is the difference between control flags and the status flags?

- A flag register stores the results produced by the execution of an instruction or controls certain operations of the EU.
- In 8086, the Execution Unit (EU) contains a **16 bit flag register**.
- Out of the 16 bits **9 are active flags** and remaining **7 are Unused (U)**.
- Out of the 9 active flags
 - **6 Status Flags** – Sign Flag(SF), Zero Flag(ZF), Auxiliary Carry Flag(AC), Parity Flag(PF), Carry Flag(CF), Overflow Flag (OF)
 - **3 Control Flags** – Direction Flag(DF), Interrupt Flag (IF), Trap Flag (TF)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
U	U	U	U	OF	DF	IF	TF	SF	ZF	U	AF	U	PF	U	CF

Status Flags

Flag	Purpose
Carry (CF)	Holds the carry after addition or the borrow after subtraction. Also indicates some error conditions, as dictated by some programs and procedures .
Parity (PF)	PF=0;odd parity, PF=1;even parity.
Auxiliary (AF)	Holds the carry (half – carry) after addition or borrow after subtraction between bit positions 3 and 4 of the result (for example, in BCD addition or subtraction.)
Zero (ZF)	Shows the result of the arithmetic or logic operation. Z=1; result is zero. Z=0; The result is 0
Sign (SF)	Holds the sign of the result after an arithmetic/logic instruction execution. S=1; negative, S=0 positive.
Overflow (OF)	Overflow occurs when signed numbers are added or subtracted. An overflow indicates the result has exceeded the capacity of the Machine.

Control Flags

Flag	Purpose
Trap (TF)	Enables the trapping through an on-chip debugging feature. <i>Permits operation of the processor in single step mode.</i>
Interrupt (IF)	Controls the operation of the INTR (interrupt request) I=0; INTR pin disabled. I=1; INTR pin enabled. <i>Indicates whether external interrupts are being processed or ignored</i>
Direction (DF)	It selects either the increment or decrement mode for DI and /or SI registers during the string instructions. <i>Indicates left or right for moving or comparing string data.</i>

- **Status Flags** reflect properties of the last executed arithmetic or logical instruction.
- For example, if register **AL = 7Fh** and the instruction **ADD AL,1** is executed then the following happen

AL = 80h

CF = 0; there is no carry out of bit 7

PF = 0; 80h has an odd number of ones

AF = 1; there is a carry out of bit 3 into bit 4

ZF = 0; the result is not zero

SF = 1; bit seven is one

OF = 1; the sign bit has changed

After any operation, if D[6] generates any carry and passes to D[7] OR if D[6] does not generate carry but D[7] generates, overflow flag becomes set, i.e., 1. If D[6] and D[7] both generate carry or both do not generate any carry, then overflow flag becomes reset, i.e., 0.

****If the XOR of carry-in and carry-out at D[7] comes to be 0 ,OF is set to zero.**

Q4. For the following:

(a) $97 + 48$

(c) $99 - 33$

(d) $33 - 99$

(e) $-29 + -32$

(f) $-41 - 95$

Is a two's complement overflow possible during subtraction

What will be the result and what will be the nature of the result in terms (All operations should be done using sign- magnitude form of representation)

1. is the result zero
2. is there a carry
3. is there an auxiliary carry
4. is the result negative
5. is there an two's complement overflow
6. Is the result even or odd parity

- 97 + 48

0110 0001

0011 0000

1001 0001

SF=1, CF=0, Of=1, Af=0, PF=0

- 99 - 33

0110 0011

1101 1111

0100 0010

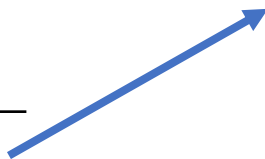
SF=0, CF=1, Of=0, Af=1, PF=1

- 33 - 99

0010 0001

1001 1101

1011 1110



Binary	1011 1110
1's Complement	0100 0001
2's Complement	0100 0010

SF=1, CF=0, Of=0, Af=0, PF=1

Decimal	33
Binary	0010 0001
1's Complement	1101 1110
2's Complement	1101 1111

Decimal	99
Binary	0110 0011
1's Complement	1001 1100
2's Complement	1001 1101

Sign bit **1** indicates that the result is negative. Take 2's complement to obtain the magnitude = 66.

• -29 + -32

1110 0011

1110 0000

1100 0011

Binary	1100 0011
1's Complement	0011 1100
2's Complement	0011 1101

Decimal	29
Binary	0001 1101
1's Complement	1110 0010
2's Complement	1110 0011

Decimal	32
Binary	0010 0000
1's Complement	1101 1111
2's Complement	1110 0000

Sign bit **1** indicates that the number is negative. Take 2's complement to obtain the magnitude = 61.

SF=1, CF=1, Of=0, Af=0, PF=1

• -41 - 95

1101 0111

1010 0001

0111 1000

Decimal	41
Binary	0010 1001
1's Complement	1101 0110
2's Complement	1101 0111

Decimal	95
Binary	0101 1111
1's Complement	1010 0000
2's Complement	1010 0001

SF=0, CF=1, Of=1, Af=0, PF=1

Q5. Is there a possibility of two's complement overflow during subtraction?

- Yes.

Eg:

Two's Complement overflow was observed in case of -41-95

1101 0111

1010 0001

0111 1000

SF=0, CF=1, Of=1, Af=0, PF=1

Decimal	41	Decimal	95
Binary	0010 1001	Binary	0101 1111
1's Complement	1101 0110	1's Complement	1010 0000
2's Complement	1101 0111	2's Complement	1010 0001